

Outil de surveillance de l'état du terrain d'un hippodrome

Table des matières

1 – Situation dans le projet.....	2
1.1 - Description de la partie personnelle.....	2
2 – Fonctionnement global logique de l'application.....	2
2.1 – Demande des données à l'API.....	4
2.2 – Fonctionnement des Json en C# .NET.....	5
3 – Ergonomie de l'interface utilisateur.....	6
3.1 – Objectif ergonomique.....	6
3.2 – Choix de conception.....	6
3.3 – Implémentation technique : GroupBox, TableLayoutPanel et Dock.....	7
3.4 – Résultat obtenu.....	7
4 - Calcul de l'évapotranspiration.....	7
4.1 - Objectif Fonctionnel.....	7
4.2 - Les avantages de la formules de Penman Monteith.....	8
4.3 - Implémentation dans le programme.....	9
4.4 - Tests unitaires.....	11
4.4.1 - Test unitaire de la classe CEvapotranspiration.....	11
4.4.2 - Problèmes rencontrés.....	13
5 – Visualisation des données météorologiques par des courbes.....	14
5.1 – Objectif fonctionnel.....	14
5.2 – Composant utilisé : Chart de WinForms.....	14
5.3 – Utilisation principale du Chart.....	14
5.4 – Implémentation technique des différents événements du Chart.....	15
5.5 – Bilan sur la courbe.....	17
6 - Bilan de la réalisation personnelle.....	17

1 – Situation dans le projet

1.1 - Description de la partie personnelle

Ma partie consiste à afficher plusieurs données à l'aide d'une IHM.

Les données météorologiques (Température de l'air, Hygrométrie, Pluviométrie, Evapotranspiration, Direction du vent et Vitesse du vent) doivent être affichées à travers plusieurs moyens. C'est à dire en texte avec la valeur la plus récente connue, mais aussi à travers des courbes s'allongeant sur le temps.

On doit aussi afficher le dernier arrosage pour toute les pistes, ainsi que montrer un historique des arrosages à travers des courbes.

Enfin, afficher les données sur les conditions du terrain (humidité du sol et température du sol), par exemple, grâce à une carte.

La donnée sur l'ensoleillement est accessible mais n'a pas besoin d'être affichée, cependant, elle est nécessaire pour le calcul de l'évapotranspiration.

L'obtention de ces données seront à travers des demandes sur une API.

2 – Fonctionnement global logique de l'application

L'application est composé de plusieurs classes ayant chacune leur rôle

Pour ce qui est de la classe frontière, comme nous faisons un projet WinForm nous allons utiliser la classe Form, où l'on va utiliser les différents événements pour interpréter les stimuli de l'opérateur, que ça soit des clic de bouton pour actualiser des dates, l'utilisateur qui survole avec sa souris un contrôle, ou la sélection de texte dans une combo box pour afficher d'autre valeur.

Pour ce qui est des classes entités, il faut savoir quoi stocker. Prenons l'exemple de la température de l'air, il faudrait que l'on sache la valeur de la température de l'air, mais aussi son horodatage (quand a-t-elle été prélevée?) ainsi que le type de donnée à laquelle la valeur se réfère (ici, la température de l'air), cependant, si l'objet demandant la température de l'air ne peut que demandé ce type de donnée et la stocker dans ces attributs, alors la dernière information (le type de la donnée) n'est plus forcément nécessaire, ça permet d'éviter des informations redondantes et donc de ne pas demander des données superflues inutilement. Donc toute données pour le moment a besoin au moins d'un champs pour la valeur et un champs pour sa date de prélèvement.

La plupart des données peuvent se contenter de uniquement 2 champs (valeur et date), on peut implicitement savoir leur type en sachant quel objet les a demandés :

- Evapotranspiration
- Ensoleillement
- Température de l'air
- Hygrométrie
- Pluviométrie
- Direction du vent
- Vitesse du vent

Cependant, certaines données ont besoin de plus de deux champs.

1. L'arrosage a besoin d'une valeur permettant de savoir combien de litre ont été écoulé pour l'arrosage, ainsi qu'une date, mais il a besoin en plus d'une localisation, un identifiant de la piste pourra servir comme point de repaire
2. La température du sol et l'humidité du sol ont le même problème, il leur faut un champs précisant leur zone.

En reprenant l'exemple de la température de l'air, pour la programmation, on pourrait utiliser deux tableaux, un en float représentant la valeur et un autre tableau DateTime représentant la date de prélèvement, cependant, comme il est prévu de recevoir des informations sous forme de json, il serait plus judicieux de faire de même sur la partie code, on va donc utiliser la référence à System.Text.Json
Cet espace de nom peut notamment créer des classes agissant comme des JSON, avec le principe de clef et de valeur. Il peut aussi convertir un string en objet JSON.

Pour notre cas, on peut discerner trois types de JSON différent :

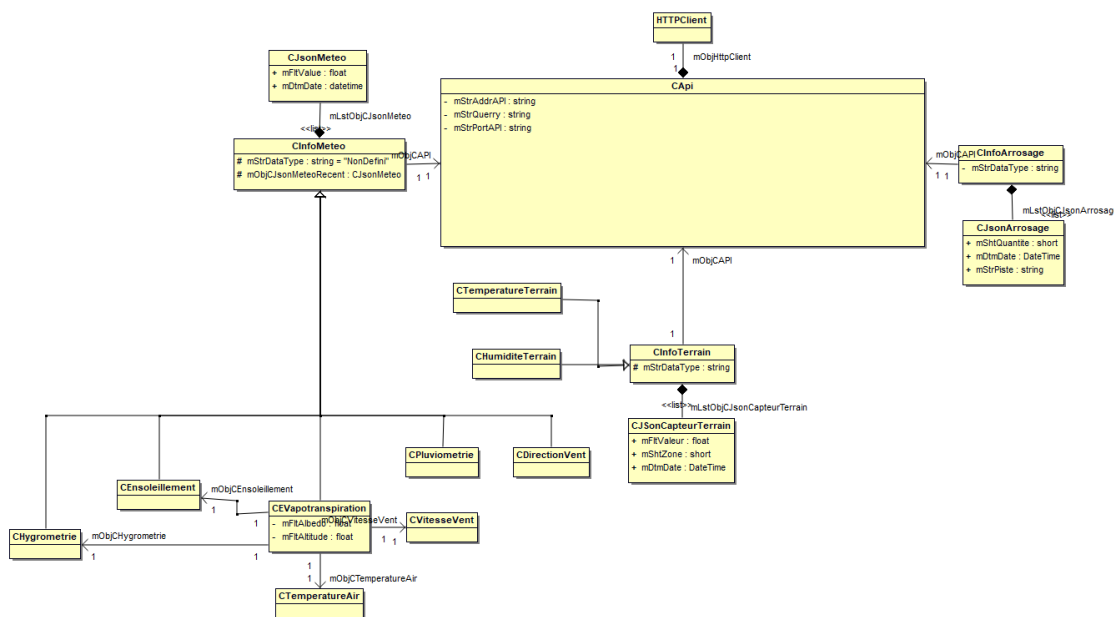
1. JSON pour la météo avec comme clefs valeur et date
2. JSON pour l'arrosage avec comme clefs quantité, date et piste
3. JSON pour les données de terrains avec comme clefs valeur, date et zone

Les classes auront comme nom CJson suivi du type de la données qu'ils stockent

Pour ce qui est des classes contrôles. Il faudrait de quoi communiquer à l'API, de quoi exploiter les données pour qu'elle puisse être affichées

Il faut une classe pour communiquer avec l'API ainsi qu'une classe par données. Les classes exploitant les même type de données (météo comme terrain) vont hériter d'une même classe plus haute permettant d'exploiter les données stockées dans les JSON, cependant, la classe parent ne sera pas directement capable de demander des données, seul ses héritiers le pourront.

Avec toute ces informations on peut en sortir un diagramme de classe.
Pour le rendre plus lisible, les méthodes ne sont pas affichées :



HttpClient est une classe permettant de faire des requêtes HTTP.

Pour ce qui est de la classe CEvapotranspiration, ses associations et son fonctionnement seront expliqués plus tard de ce document.

Le fonctionnement du système va être le suivant, les objets CInfoMeteo, CInfoTerrain et CInfoArrosage vont devoir interagir avec un objet CApi.

Pour recevoir les bons types de données, l'objet utilisant CApi précisera en paramètre son attribut mStrDataType, cet attribut n'a pas de set et sert à dire quel est le type de données qui nous intéresse.

Fabien DAVID

Par exemple, CInfoMeteo peut interagir avec un objet CApi mais n'a pas une valeur valide pour son attribut mStrDataType, cependant, ces héritiers auront une valeur valide et prédéfinie.

C'est le même principe pour CinfoTerrain.

Une fois que l'objet CApi retourne les données demandées, elles pourront être stockées sur une liste CJson composante de la classe ayant fait la demande à CApi.

Les deux sous parties vont aller plus en détails dans comment faire des demandes à l'API et comment exploiter son renvoi.

2.1 – Demande des données à l'API

Les demandes à l'API seront faites via le protocole HTTP

N'ayant pas de DNS sur le réseau, il faudra utiliser l'IP et le port de l'API pour pouvoir communiquer avec elle

La classe CApi a donc en attribut privé un string pour l'IP et le port de l'API.

Pour pouvoir envoyer des requêtes HTTP, le framework .Net propose la classe HttpClient.

Elle a plusieurs méthodes permettant de faire des GET et POST HTTP.

On va s'intéresser au GET HTTP. Le POST est intéressant puisque l'envoi de données sur les arrosages des pistes devrait être possible à partir de l'application, cependant cela n'a pas été fait.

Le GET permet de recevoir des données de l'API.

Deux méthodes de CApi vont nous intéresser

La première est mVFunWriteDdeQuery :

```
public void mVFunWriteDdeQuery(  
    String strTargetData,  
    DateTime dtmDateDeDebut,  
    DateTime dtmDateDeFin  
)  
{  
    mStrQuery = String.Concat(  
        mStrAddrAPI,  
        ":",  
        mStrPortAPI,  
        "/",  
        strTargetData,  
        "&DateDeDebut=",  
        dtmDateDeDebut.ToString(),  
        "&DateDeFin=",  
        dtmDateDeFin.ToString()  
    );  
    return;  
}
```

Cette méthode accepte 3 paramètres :

1. strTargetData : string qui est la donnée ciblée. Elle accepte des valeurs qui sont en corrélation avec les attributs mStrDataType de CInfoMeteo, CInfoArrosage et CinfoTerrain
2. dtmDateDeDebut : DateTime, Ce paramètre prendra la valeur du DateTimePicker du Form représentant la date de début souhaitée

Fabien DAVID

3. dtmDateDeFin : DateTime, Ce paramètre prendra la valeur du DateTimePicker du Form représentant la date de fin souhaitée

Partie du code de mStrFunGetJsonQuery

```
string response;

try
{
    using (HttpResponseMessage httpResponse = await mObjHttpClient.GetAsync(mStrQuery))
    {
        httpResponse.EnsureSuccessStatusCode();
        response = await httpResponse.Content.ReadAsStringAsync();
    }
}
catch (HttpRequestException e)
{
    Console.WriteLine("\nException ");
    Console.WriteLine("Message :{0} ", e.Message);
}

return response;
}
```

La ligne « mObjHttpClient.GetAsync(mStrQuery) » veut dire que l'on fait une demande GET à l'url passé en paramètre.

La méthode d'avant formait l'url et le mettait dans l'attribut mStrQuery

Grace à cette méthode, on peut donc faire des demandes à l'API

2.2 – Fonctionnement des Json en C# .NET

Le framework .NET propose de pouvoir traiter des Json avec l'espace de nom System.Text.Json. Il permet d'avoir des objets Json, et donc d'avoir des valeurs à partir de leurs clefs

Voici la syntaxe de la classe CJsonMeteo

```
public class CJsonMeteo
{
    [JsonPropertyName("valeur")]
    public float mFltValeur { get; set; }

    [JsonPropertyName("date")]
    public DateTime mDtmDate { get; set; }
}
```

Concrètement, on indique qu'un attribut découle d'une clef utilisé pour un json.

Donc quand un Json aura une clef « valeur », elle sera traduite dans l'attribut mFltValeur

```
new List<CJsonMeteo> (JsonSerializer.Deserialize<List<CJsonMeteo>> (strDonneesAConvertir))
```

Cette commande permet de créer une liste d'objet Json à partir d'un texte passé en paramètre. Grace à la clef « Date » dans le Json, on va pouvoir faire en sorte qu'ils aient un ordres chronologiques

Dans notre cas, le premier élément du tableau aura la Date la plus récente.
Donc le dernier élément du tableau aura la Date la plus ancienne.

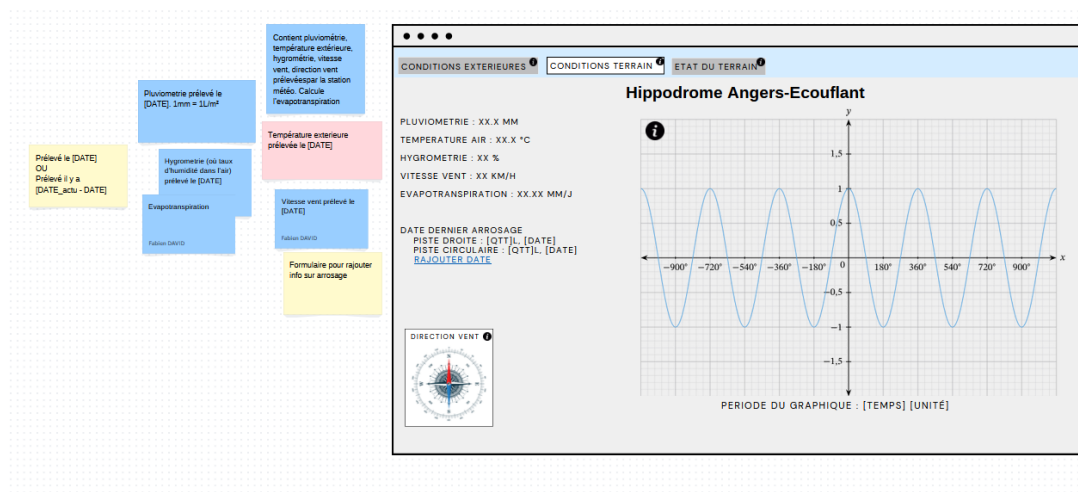
3 – Ergonomie de l'interface utilisateur

3.1 – Objectif ergonomique

L'interface utilisateur doit permettre une consultation rapide, fluide et structurée des données météorologiques.

L'objectif ergonomique est double :

- Offrir un aperçu immédiat des valeurs les plus récentes, sans avoir besoin d'interagir avec l'application
- Permettre une analyse plus poussée à travers des visualisations graphiques comme les courbes et les histogrammes.



Voici le schéma conceptuel de l'IHM. Celui ci a changé au fur et a mesure du développement mais garde les mêmes idées de base.

Les valeurs récentes sont affichées à gauche, et un graphique se trouve sur la droite.

3.2 – Choix de conception

Les dernières valeurs des indicateurs météo (température de l'air, hygrométrie, pluviométrie, vitesse et direction du vent, évapotranspiration) sont affichées sous forme textuelle, dans un panneau fixe situé à gauche de l'écran, une icône à gauche du texte aide à préciser le type de donnée.

Chaque valeur est associée à un label, accompagné d'un tooltip expliquant l'origine ou la méthode de calcul de la donnée.

À droite de l'écran se trouve la zone graphique, dédiée à la visualisation des historiques via des courbes (type Chart).

Fabien DAVID

L'utilisateur peut sélectionner la donnée à afficher, zoomer, ou cliquer sur les zones à gauche pour faire apparaître la courbe correspondante.

La zone graphique à sa propre partie plus tard dans ce document

3.3 – Utilisation de GroupBox, TableLayoutPanel et Dock

Afin d'assurer une organisation claire et une mise en page fluide, l'interface repose sur les composants suivants :

- GroupBox, permet de regrouper visuellement les informations liées. Par exemple, le graphique est dans un GroupBox avec ces différents boutons.
- TableLayoutPanel, assure une distribution ordonnée en lignes et colonnes, facilitant l'alignement des textes, des champs de saisie ou des boutons. Il est utilisé dans la partie de gauche de l'ihm, pour aligner texte et logo
- La propriété Dock est utilisée pour organiser dynamiquement l'espace
 - Le TableLayoutPanel sur les valeur récentes est docké à gauche
 - La GroupBox regroupant les graphiques est docké à droite

Ce système d'agencement automatique offre plusieurs bénéfices :

- L'interface s'adapte au redimensionnement de la fenêtre sans déformer les éléments
- La séparation entre valeurs statiques et graphiques est claire et stable
- Il évite l'utilisation de coordonnées manuelles, ce qui améliore la maintenabilité de l'interface.

4 - Calcul de l'évapotranspiration

4.1 - Objectif Fonctionnel

L'évapotranspiration est une donnée qui doit être affichée sur l'IHM.

Elle indique quantitativement la « transpiration » du sol.

L'évapotranspiration est essentiellement l'eau contenue dans le sol qui se vaporise. L'eau vaporisée quitte le sol et retourne dans l'atmosphère. Donc plus l'évapotranspiration est élevée et plus le sol risque d'être sec dans un futur proche, il est donc important de savoir la valeur actuelle de l'évapotranspiration.

L'évapotranspiration est en mm/j. Pour une évapotranspiration égale à X mm/j, Il y aura X litre d'eau par mètre carré par jour qui vont se vaporiser. Donc sur de grand terrain (comme un hippodrome), il est important de savoir combien de litre d'eau vont être vaporisé du sol dans un futur proche pour pouvoir anticiper un arrosage.

4.2 - Les avantages de la formules de Penman Monteith

Nous n'avons pas de capteur d'évapotranspiration, cependant plusieurs formules sont valide pour la calculer, on s'intéressera qu'à celle de Penman-Monteith. La formule de Penman-Monteith requiert des paramètres qui sont à notre disposition grâce à la station météo.

La formule de Penman-Monteith est la suivante :

$$EPT = \frac{(0,408 * \Delta * R_{gnet} + \gamma * (\frac{900}{(T_{Moy} + 273)}) * u^2 * (es - ea))}{(\Delta + \gamma * (1 + 0,34 * u^2))}$$

Les paramètres nécessaires sont :

Fabien DAVID

1. U_2 : Vitesse du vent à 2m du sol sur une période (m/s)
2. T_{moy} : Température moyenne sur une période (°C)
3. HR_{moy} : Humidité relative moyenne sur une période (%)
4. R_g : Rayonnement ($MJ/m^2/j$)
5. Albedo : $\sigma = 0,23$
6. Altitude : $z = 48,6$ (m)

Les paramètres 1 à 4 sont disponibles grâce à la station météo.

Les paramètres 5 et 6 sont déduisables.

La période minimale (vis à vis des moyennes) pour calculer l'évapotranspiration (ou EPT) est de 1 jour.

La vitesse du vent peut être prélevée à 10m du sol au lieu de 2m, une conversion est possible pour passer d'une valeur à l'autre.

$$U_2 = U_{10} * 0.7$$

L'albédo est la part des rayonnement solaires qui sont renvoyées vers l'atmosphère. Elle a une valeur entre 0 et 1. Si l'albédo est de 0, ça veut dire que tout les rayons du soleil sont absorbés. A l'inverse, si l'albédo est de 1, ça veut dire que tout les rayons du soleil sont réfléchis sur cette surface. Un exemple dans la vraie vie est la neige, la couleur blanche n'absorbant pas ou peu la lumière, la neige a un albédo de environ 0,8.

Pour notre cas, un hippodrome, donc un terrain de gazon bien entretenu et souvent arrosé a un albédo de environ 0,23. N'ayant pas en possession un capteur pouvant mesurer l'albédo en tout temps, nous allons devoir nous contenter de cette estimation.

Cette valeur pourra être modifiable dans le programme, elle sera l'un des attributs de CEvapotranspiration.

L'altitude pourra changer selon l'hippodrome, celui d'Angers-Ecouflant a une altitude de 48,6m.

Cette valeur pourra être modifiable dans le programme, elle sera l'un des attributs de CEvapotranspiration.

$$EPT = \frac{(0,408 * \Delta * R_{gnet} + \gamma * (\frac{900}{(T_{Moy} + 273)}) * u^2 * (e_s - e_a))}{(\Delta + \gamma * (1 + 0,34 * u^2))}$$

Vitesse du vent à 2m (m/s) : $U_2 = U_{10} * 0,7$

Rayonnement net ($MJ/m^2/j$) : $R_{gnet} = \text{rayonnement}(1 - \text{albédo})$

Pression vapeur saturée (kPa) : $e_s = 0,6108 * e^{((17,27 * T_{moy}) / (273,3 + T_{moy}))}$

Pression vapeur effective (kPa) : $e_a = e_s * HR_{moy} / 100$

Pente de la courbe pression vapeur saturée (kPa/°C) : $\Delta = \frac{4098 * e_s}{((237,3 + T_{moy})^2)}$

Pression atmosphérique (kPa) : $P = 101,3 * ((\frac{(293 - 0,0065 * z)}{293})^{5,26})$

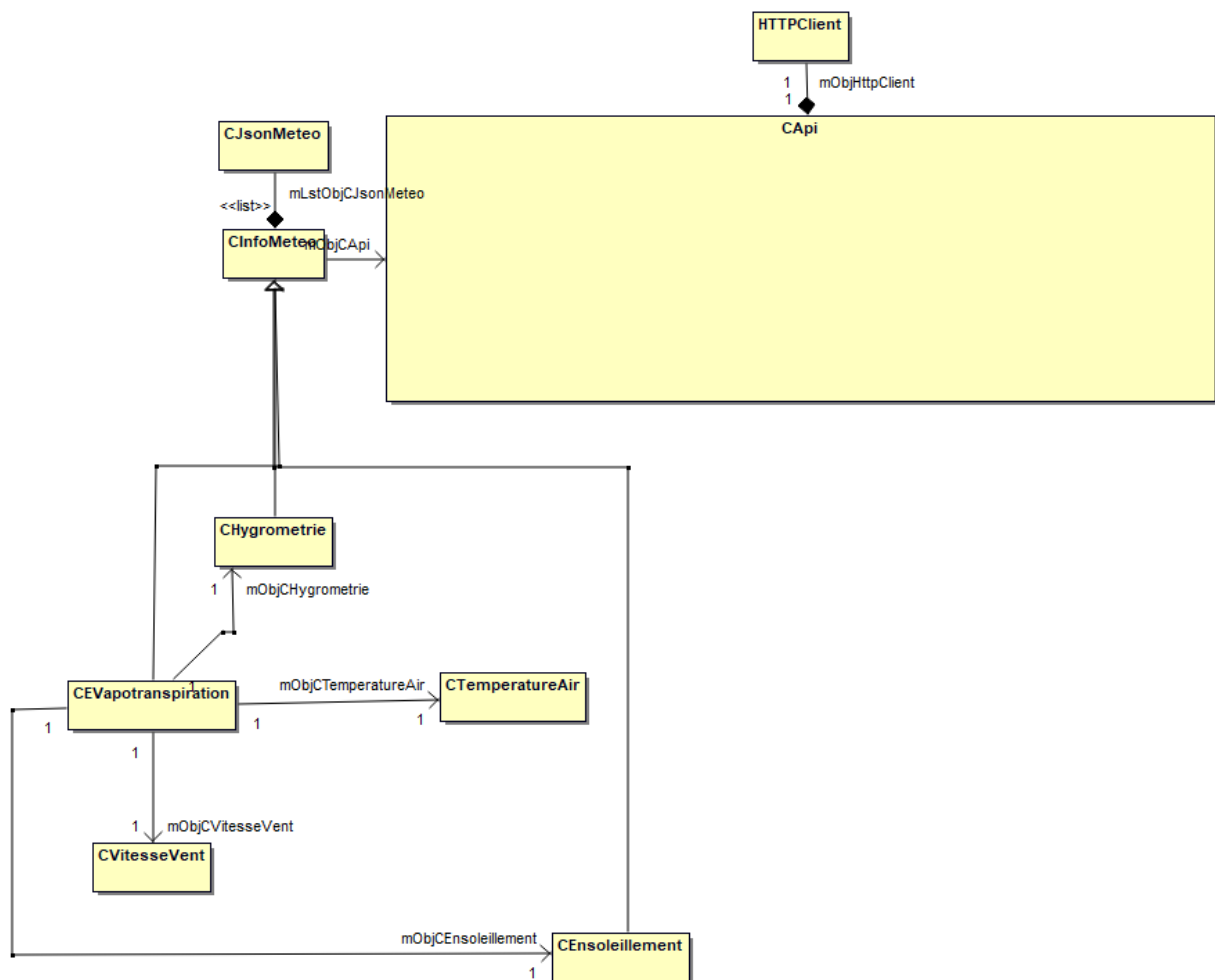
Constante psychométrique (kPa/°C) : $\gamma = 0,665 * P * 10^{-3}$

La pression de vapeur saturée est la pression à laquelle la phase gazeuse de l'eau est en équilibre avec sa phase liquide ou solide à une température donnée

Encore une fois, l'albédo n'étant pas prélevé par des capteurs nous oblige à l'estimer. Cependant, tout les autres paramètres étant prélevés par des capteurs ou sont connus, le calcul de l'évapotranspiration est donc fiable.

4.3 - Implémentation dans le programme

Dans le programme, la classe s'occupant de ce type de donnée est CEvapotranspiration, voici le diagramme de classe avec les classes pertinentes



CEVapotranspiration
- mFitAlbedo : float - mFitAltitude : float + <<list>> mLstObjCJsonMeteoMoyenneTemperatureAir : CJsonMeteo + <<list>> mLstObjCJsonMeteoMoyenneHygrometrie : CJsonMeteo + <<list>> mLstObjCJsonMeteoMoyenneEnsoleillement : CJsonMeteo + <<list>> mLstObjCJsonMeteoMoyenneVitesseVent : CJsonMeteo
+ CEvapotranspiration() + CEvapotranspiration(in objCAPI : CApi) + CEvapotranspiration(in objCAPI : CApi, in objCTemperatureAir : CTemperatureAir, in objCHygrometrie : CHygrometrie, in objCVitesseVent : CVitesseVent, in objCEnsoleillement : CEnsoleillement) + ~CEvapotranspiration() + mVFunSetAssoCEnsoleillement(in objCEnsoleillement : CEnsoleillement) : void + mVFunSetAssoCHygrometrie(in objCHygrometrie : CHygrometrie) : void + mVFunSetAssoCTemperatureAir(in objCTemperatureAir : CTemperatureAir) : void + mVFunSetAssoCVitesseVent(in objCVitesseVent : CVitesseVent) : void + mFitCalculEvapotranspiration(in fitMoyenneTemperatureAir : float, in fitMoyenneHygrometrie : float, in fitMoyenneEnsoleillement : float, in fitMoyenneVitesseVent : float) : float + mFitFunGetAlbedo() : float + mVFunSetAlbedo(in fitAlbedo : float) : void + mFitFunGetAltitude() : float + mVSetAltitude(in fitAltitude : float) : void + mVFunSetmLstObjCJsonMeteoMoyenneTemperatureAir(in strDonneesACconvertir : string) : void + mVFunSetmLstObjCJsonMeteoMoyenneHygrometrie(in strDonneesACconvertir : string) : void + mVFunSetmLstObjCJsonMeteoMoyenneEnsoleillement(in strDonneesACconvertir : string) : void + mVFunSetmLstObjCJsonMeteoMoyenneVitesseVent(in strDonneesACconvertir : string) : void + mVFunUpdatemLstObjCJsonMeteo(in dtmDateDeDebut : DateTime, in dtmDateDeFin : DateTime) : void + mVFunUpdatemLstObjCJsonMeteoMoyenneTemperatureAir(in dtmDateDeDebut : DateTime, in dtmDateDeFin : DateTime) : void + mVFunUpdatemLstObjCJsonMeteoMoyenneHygrometrie(in dtmDateDeDebut : DateTime, in dtmDateDeFin : DateTime) : void + mVFunUpdatemLstObjCJsonMeteoMoyenneEnsoleillement(in dtmDateDeDebut : DateTime, in dtmDateDeFin : DateTime) : void + mVFunUpdatemLstObjCJsonMeteoMoyenneVitesseVent(in dtmDateDeDebut : DateTime, in dtmDateDeFin : DateTime) : void

La classe CApi sert à communiquer avec l'API.

La classe CJsonMeteo sert à stocker les données renvoyées par un objet CApi

La classe CInfoMeteo sert à utiliser un objet CApi pour demander des informations sur son attribut mStrDataType et les stocker dans une liste de CJsonMeteo. Le mStrDataType de CInfoMeteo ne lui permet pas de faire des demande mais sera modifié dans les constructeurs de ces héritiers.

La classe CHygrometrie peut exploiter les informations sur l'hygrométrie

La classe CEnsoleillement peut exploiter les informations sur l'ensoleillement

La classe CTemperatureAir peut exploiter les informations sur la température de l'air

La classe CVitesseVent peut exploiter les informations sur la vitesse du vent

CEvapotranspiration ne peut pas demander les données à l'API puisqu'il n'y a pas un capteur fournissant la donnée de l'évapotranspiration, à la place, il doit faire le calcul grâce aux données fournies par les 4 héritiers de CInfoMeteo mentionné plus haut. Pour ce faire, il va faire les calculs de la formule de Penman Monteith en prenant en paramètres les moyennes journalières de température, hygrométrie, vitesse du vent et ensoleillement. Ces moyennes sont retrouvables dans ces attributs mLstObjCJsonMeteoMoyenne suivit du type de données concerné.

4.4 - Tests unitaires

Nous allons voir les tests unitaires sur CEvapotranspiration, à la partie 3 du document « TU_Etudiant3.odt »

4.4.1 - Test unitaire de la classe CEvapotranspiration

Élément testé :		CEvapotranspiration.mFItFunCalculEvapotranspiration(float,float,float,float) : 3.1		
Objectif du test :		Savoir si le calcul de l'évapotranspiration est fidèle et savoir si certaines valeurs aberrantes permettent vont empêcher le calcul de l'évapotranspiration		
Nom du testeur :		Fabien DAVID		Date : 30/04/2025
Moyens mis en œuvre :		Logiciel : ApplicationSurveillanceHippodrome	Matériel :	Outil de développement : VisualStudio2022
Procédure du test : On teste la méthode CEvapotranspiration.mFItFunCalculEvapotranspiration(float,float,float,float). Si la méthode retourne -1, ça signifie qu'une erreur s'est produite, en effet, l'évapotranspiration ne peut pas être négative				
Id	Description du vecteur de test	Résultat attendu	Résultat obtenu	Validation (O/N)
3.1.1	1. Instancier un objet CEvapotranspiration nommé objCEvapotranspiration 2. lui faire faire la méthode objCEvapotranspirationm.mFItFunCalculEvapotranspiration(25F, 50F, 12F, 20F)	10.00	10.00	O
3.1.2	Instancier un objet CEvapotranspiration nommé objCEvapotranspiration lui faire faire la méthode objCEvapotranspirationm.mFItFunCalculEvapotranspiration(-21F, 50F, 12F, 20F) objCEvapotranspirationm.mFItFunCalculEvapotranspiration(81F, 50F, 12F,	-1	-1	O

	20F) Les deux résultats doivent être -1			
3.1.3	Instancier un objet CEvapotranspiration nommé objCEvapotranspiration lui faire faire la méthode objCEvapotranspirationm.mFltFunCalculEvapotranspiration(25F, -1F, 12F, 20F) objCEvapotranspirationm.mFltFunCalculEvapotranspiration(25F, 101F, 12F, 20F) Les deux résultats doivent être -1	-1	-1	0
3.1.4	Instancier un objet CEvapotranspiration nommé objCEvapotranspiration lui faire faire la méthode objCEvapotranspirationm.mFltFunCalculEvapotranspiration(25F, 50F, 12F, -1F) objCEvapotranspirationm.mFltFunCalculEvapotranspiration(25F, 50F, 12F, 61F) Les deux résultats doivent être -1	-1	-1	0
Conclusion du test :		Le calcul de l'évapotranspiration est fiable, de plus, il renvoi -1 quand les paramètres d'entrées sont aberrants. Le test est réussi		

Fabien DAVID

Le prochain procès verbal présente la partie 3.2, cependant les parties 3.3, 3.4 et 3.5 sont similaires

Éléments testés :	CEvapotranspiration.mVFunSetmLstObjCJsonMeteoMoyenneTemperatureAir(str) : 3.2				
Objectif du test :	Toutes ces méthodes sont des set d'attributs différents. Ils convertissent un string passé en paramètre en une liste CJsonMeteo. Il faut donc être sur que le string soit convertissable Le constructeur instancie de base ces liste, leur premières date sont DateTime.MinValue, si on les retrouve encore, ça veut dire qu'elle n'ont pas été effacée, donc que la méthode n'a pas accepté le paramètre				
Nom du testeur :	Fabien DAVID		Date :	30/04/2025	
Moyens mis en œuvre :	Logiciel : ApplicationSurveillanceHippodrome	Matériel :	Outil de développement : VisualStudio2022		
Procédure du test :					
Id	Description du vecteur de test		Résultat attendu	Résultat obtenu	Validation (O/N)
3.2.1	1. Instancier un objet CEvapotranspiration nommé objCEvapotranspiration 2. lui faire faire la méthode testée en ajoutant en paramètre le fichier TestBonJson.json 3. Ensuite, afficher la valeur avec objCEvapotranspiration.mLstObjCJsonMeteoMoyenneTemperatureAir[1].mFltValeur		14	14	O
3.2.2	1. Instancier un objet CEvapotranspiration nommé objCEvapotranspiration 2. Faire la fonction testée en passant en paramètre le fichier TestMauvaisJson.Json 3. Ensuite, afficher la valeur avec objCEvapotranspiration.mLstObjCJsonMeteoMoyenneTemperatureAir[0].mDtmDate		DateTime.MinValue	DateTime.MinValue	O
Conclusion du test :		Les conditions acceptant la conversion du string en liste CJsonMeteo sont acceptables			

4.4.2 - Problèmes rencontrés

Pour les tests de 3.2 à 3.5 le paramètre normal passé en paramètre n'était pas sous un bon format, entraînant un échec automatique du test. Ceci a été changé par la lecture d'un fichier json pour simplifier les tests et éviter de répéter les mêmes erreurs

5 – Visualisation des données météorologiques par des courbes

5.1 – Objectif fonctionnel

Afin de permettre au client de mieux analyser l'évolution des conditions météorologiques dans le temps, l'interface doit proposer un affichage clair, interactif et précis de ces informations.

Une représentation graphique par courbes temporelles s'impose naturellement pour visualiser des historiques tels que la température de l'air, l'hygrométrie, la pluviométrie, l'évapotranspiration et la vitesse du vent.

Cependant, une simple courbe linéaire ne suffit pas à l'analyse. L'utilisateur doit pouvoir zoomer dynamiquement sur une plage de temps plus précise. De plus, passer sa souris sur un point doit lui donner des renseignements supplémentaires.

Il faut donc l'ajout d'une fonction de zoom interactif, via :

- La molette de la souris pour un zoom progressif autour du curseur
- Une sélection de zone à la souris pour délimiter un intervalle temporel

5.2 – Composant utilisé : Chart de WinForms

L'affichage des courbes est réalisé dans une zone spécifique de l'IHM, elle se trouve à droite quand on est dans la page condition météo de l'application. Cette zone utilise un composant Chart du namespace System.Windows.Forms.DataVisualization.Charting, ce namespace est intégré dans .NET.

Le composant Chart permet :

- De gérer plusieurs séries de données (dont météorologiques)
- D'afficher les points sur un graphique (de type Line)
- D'autoriser des interactions utilisateur (comme le zoom et la visualisation par survol)

5.3 – Utilisation principale du Chart

La courbe météo repose sur les données extraites de la classe CInfoMeteo, à travers ses héritiers (CTemperatureAir, CHygrometrie, etc.), qui utilisent tous une liste de CJsonMeteo pour stocker les points (date + valeur).

La méthode mVFunActualiserMChTmMeteo est responsable de l'affichage ou de la suppression d'une série, elle

- Récupère les données pertinentes
- Applique un filtrage temporel (entre deux dates sélectionnées)
- Lie les données au Chart avec sa méthode DataBindXY
- Applique une couleur, un style, et un format de date lisible
 - Si la période sélectionnée est contenue dans l'année actuelle (2025 pour notre cas), alors l'année ne sera pas affichée dans l'axe des abscisses, et inversement

La méthode n'est pas complexe mais est longue.

mVFunActualiserMChTmMeteo prend un paramètre un string. La valeur du string va définir la couleur de la courbe ainsi que les données à afficher / enlever, ces données sont trouvables dans les attributs de ApplicationHippodrome qui héritent de CInfoMeteo.

Les données sont obtenables par la méthode

CInfoMeteo.mLstObjCJsonMeteoFunGetmLstObjCJsonMeteo(), on prend toute les données étant dans la période demandées puis on les affiche sur mChTmMeteo.

Fabien DAVID

Ce mécanisme est indépendant du zoom, mais parfaitement compatible avec lui. Une fois les courbes affichées, l'utilisateur est libre d'explorer l'historique sans rechargement de données

5.4 – Implémentation technique des différents événements du Chart

L'objet Chart permet d'afficher les graphiques

Un Chart est composé de ChartAreas

L'objet Chart dans mon programme s'appelle mChtMeteo

Les lignes suivantes permettent d'autoriser les fonctionnalités de zoom sur mChtMeteo

Elles sont exécutées lors de l'instanciation de l'objet Form qui sert à afficher l'IHM

Partie du code du constructeur de classe ApplicationHippodrome : Form

```
// Active le zoom par sélection sur l'axe X du graphique
mChtMeteo.ChartAreas[0].CursorX.IsUserEnabled = true;
mChtMeteo.ChartAreas[0].CursorX.IsUserSelectionEnabled = true;
mChtMeteo.ChartAreas[0].AxisX.ScaleView.Zoomable = true;

// Idem sur l'axe Y du graphique :
mChtMeteo.ChartAreas[0].CursorY.IsUserEnabled = true;
mChtMeteo.ChartAreas[0].CursorY.IsUserSelectionEnabled = true;
mChtMeteo.ChartAreas[0].AxisY.ScaleView.Zoomable = true;
```

Cela permet à l'utilisateur de cliquer-glisser sur la zone de la courbe pour effectuer un zoom sur une période ou une plage de valeurs spécifique.

Pour ce qui est du zoom avec la molette de la souris, on crée la méthode mVFunmChtMeteoMouseWheel, elle est appelée lorsque l'événement MouseWheel est activé sur le graphique

Cette méthode calcule dynamiquement un facteur de zoom (de $\pm 20\%$) et applique un agrandissement ou un rétrécissement de l'axe des abscisses :

Partie du code de mVFunmChtMeteoMouseWheel :

```
// On convertit la position du curseur de la souris (en pixels)
// en valeur correspondante sur l'axe X (date)
double dblPosition = caChartArea.AxisX.PixelPositionToValue(e.X);

// L'affichage minimum est de une journée
double dblMinViewSize = 1; // -> 1 équivaut à une journée

if (e.Delta > 0) // Zoom avant (molette vers le haut)
{
    // On réduit la taille de la vue actuelle selon le facteur de zoom (ici 20%)
    dblNewSize = dblViewSize * (1 - dblZoomFactor);
```

```
// Si la nouvelle taille devient trop petite,  
// alors on impose une taille minimale de vue pour ne pas zoomer "trop"  
if (dblNewSize < dblMinViewSize)  
    dblNewSize = dblMinViewSize; // Empêche un zoom excessif  
  
// Calcule de la nouvelle position minimale (bord gauche du zoom),  
// en gardant le zoom centré autour de la position de la souris  
double newMin = dblPosition - (dblPosition - dblXMin) * (dblNewSize / dblViewSize);  
  
// La nouvelle position maximale est simplement newMin + la taille de la vue  
double newMax = newMin + dblNewSize;  
  
// Application du zoom sur l'axe X entre les nouvelles bornes  
caChartArea.AxisX.ScaleView.Zoom(newMin, newMax);  
}  
  
else if (e.Delta < 0) // Zoom arrière (molette vers le bas)  
{  
    // On agrandit la vue actuelle selon le facteur de zoom  
    dblNewSize = dblViewSize * (1 + dblZoomFactor);  
  
    // Même logique que pour le zoom avant : centrer autour de la souris  
    double newMin = dblPosition - (dblPosition - dblXMin) * (dblNewSize / dblViewSize);  
    double newMax = newMin + dblNewSize;  
  
    // Application du zoom arrière  
    caChartArea.AxisX.ScaleView.Zoom(newMin, newMax);  
}
```

On vérifie d'abord si la molette a été bougée vers l'avant ou l'arrière. Ensuite, on calcule le nombre de jour total à afficher après le zoom, si celui ci est trop bas, il est réajusté.

Une fois cela fait, on calcul la nouvelle coordonnée à affichée qui est en corrélation avec ou se trouvait la souris lors de l'évènement MouseWheel.

Enfin, on applique ces nouvelles coordonnées à la ChartArea pour qu'elle puissent être affichées

Enfin, le dernier ajout concerne la position de la souris sur le graphique.

La méthode mVFunmChtMeteoMouseMove s'active quand l'utilisateur bouge la souris sur mChtMeteo.

Celle ci n'a pas un code particulièrement complexe. On prend les coordonnées de la souris, si le curseur est sur un point du graphique (et donc pas sur une zone vierge) alors on prend les paramètres de ce points (valeur et date) que l'on affiche sur un tooltip

L'apparition de tooltips ajoute un confort supplémentaire : lorsque l'utilisateur survole un point de la courbe, il visualise la date précise ainsi que la valeur numérique.

5.5 – Bilan sur la courbe

Grâce à cette fonctionnalité de zoom, l'affichage des données météorologiques devient à la fois plus lisible et plus exploitable.

L'utilisateur peut isoler une période critique sans pour autant redemander des données, ou refaire les courbes. Il peut ensuite re élargir la vue pour identifier une tendance générale.

Le tout est réalisé sans composants externes, et est intuitif grâce à la gestion des différents événements.

6 - Bilan de la réalisation personnelle

En conclusion, plusieurs points comme l'affichage des données météorologiques sont faits, même si les données sont bouchonnées ...

La prochaine étape est de concevoir le système pour l'arrosage, la partie logique est faite en partie mais la partie de l'affichage n'est pas commencée

Il aurait été préférable d'avoir un vrai planning pendant tout le projet. Cette organisation aurait pu faire gagner du temps, et les jalons auraient été clair et précis.

De plus, avoir plus de communication avec les membres du projets auraient pus l'accélérer pour tout le monde, certaines solutions ont été trouvées à travers la discussion, et des ambiguïtés ont été créée par le silence, par exemple, les données demandées de l'étudiant 1 à moi.